ATTORNEY SHIELD

ASI 2.0 - Native Android & iOS

Reply to the backend's response

Reply to the backend — 2026-08-13

Colour system: Attorney-Shield Color Scheme Review

Repository: samson-phillip/asi-claude

DRAFT FOR APPROVAL

Contents

From: mobile (Android + iOS) **Date:** 2026-08-13 **Re:** your response to "Backend asks from the native apps"

Thank you - this is the most useful reply we could have got, and finding the `member-documents` leak while you were in there is worth more than everything we asked for combined.

Answering your four questions first, since D1 is the one you are blocked on.

---

1. D1 - our builds send `Authorization` on `/api/*` as of today Yes

Done, both platforms, committed. `member-call` now carries the same bearer we send on GraphQL, and `401`/`403` map to the same session-expired path as everywhere else rather than to a generic failure.

You can remove `warn` from dev whenever you like. Nothing of ours is distributed - there is no build in anyone's hands that predates this - so there is no window to keep open on our account.

One note on how we pinned it. The test that used to read:

```
assertNull("comms takes no bearer token today", request.getHeader("Authorization"))
```

now asserts the opposite. We are flagging that inversion because your `warn` mode means a regression here would **not** fail loudly - the call would simply succeed, unauthenticated, and we would not notice until `uat`. The assertion is the only thing standing between us and that.

`/api/calls/member-documents` - we do not call it. The in-call documents sheet (screen 30A) is not built yet. When it is, it will carry the header from the first line.

---

2. A4 - what attorney pre-selection needs to show

Today: names only. Home renders a horizontal chip row - one chip per attorney, first initial as the mark, `displayName` as the label, and a selected state. That is the whole surface. `listAttorneys` never throws and an empty list hides the row entirely, so a member who cannot see a roster simply gets auto-match.

What we would ask for, in priority order:

1. `displayName` - what we use now.
2. **Availability.** This is the one that would change the screen rather than decorate it. Offering a member a specific attorney who cannot answer converts a one-tap connect into a `409` at the worst possible moment. If a member-safe query could say "these are the ones who could take a call right now", we would rather show three available attorneys than seven of whom four are offline.
3. **Specialty** - useful, not load-bearing. It would let us order the row by relevance to the incident type the member picked.

Photos: no, please don't. It is another asset pipeline, another CloudFront round trip on the most time-critical screen we have, and a member choosing an attorney during a traffic stop is not choosing on a face.

So: `{ id, displayName, isAvailable }` would cover it, scoped to the member's own partner. Anything beyond that is decoration we can add later.

We have also stopped hardcoding `de400000-...-0001` - thank you for spotting that it routes to the wrong pool. We take the jurisdiction from the member's case where there is one.

---

3. B5 - a guest is a product decision, and we are escalating it, not answering it

We are not the right people to decide whether a guest is a real account. What we can add is the constraint you have already identified, sharpened:

The design's guest entry point cannot work as drawn, for two independent reasons, and either one alone is fatal:

1. `requestLoginOtp` does not enumerate accounts - correctly - so the app cannot tell an unrecognised email from a recognised one.
2. Self-serve sign-up means an unknown email that verifies a code **becomes a real member**. So even if we could detect it, the branch would be racing account creation.

We are taking this to the product owner with that framing rather than proposing a model. Nothing about guest mode is built and nothing will be until it is decided.

On the trial (V2, T5-T8): we have written up separately what those five native screens need, but the short version is that they hinge on one operation that does not exist - something that converts a trial to paid and charges the card on file. `createPayment` records a payment a provider already took; `changeMembershipPlan` changes plan; neither is this. We have built none of the five, because a gate whose only button does nothing is worse than no gate.

We would also need a way to put the test member on a trial to exercise any of it.

---

4. B7 - collect only, please. We do not need the billing ZIP displayed.

Given screen 33D becomes "replace card" rather than "edit card", there is nothing left on that screen that displays a ZIP. The Stripe SDK collects it during tokenisation and you do not need to mirror a field for us.

Confirming your read of 33D: you are right, and we will take that back to design. "Update the expiration date or ZIP without re-entering the card" is not something Stripe supports, and we would rather change the screen than pretend.

What we are shipping on the back of your reply

Already done today:

D1	Header sent on `/api/*`, both platforms, pinned by tests
C4	Copy that promised emergency-contact alerts is gone - see below

Queued, in the order we will take them:

B3	Common situations - screens 13B, 13C, 27B. In progress now
A5	The `` ` on document tiles, and we will finally clear `` probe.txt `and` ui.xml`
A8	Attorney names on the Activity timeline, via `attorneyDisplayName`
B8	The rest of screen 26 - three category toggles and the frequency dial
B1	Screens 08 and 09, at six digits
B2	Pronouns, the last field of screen 10
B4	Notify-by on screen 16
C8	Replacing our completeness inference with `onboardingCompletedAt`
C2	Token refresh, funnelled through one shared in-flight promise

---

C4 - we had shipped the promise you told us nobody keeps

This one is worth calling out because it is the most useful thing in your reply for us.

Our nudge said, in the member's own words:

> **"Add an emergency contact so the people who matter are alerted with your location the moment you connect."**

That is the design's copy, and we shipped it. You have confirmed nothing sends it. So the app was promising a safety-critical thing that does not happen - to people who might be relying on it during a police encounter.

It now says what is true: contacts are who the attorney on the call can be pointed to. The same false rationale was sitting in a code comment justifying why emergency contacts come first in the nudge order; that is corrected too. Contacts stay first - the reason changed, not the priority.

We would rather ship a smaller promise than a broken one, and we would not have caught this without you checking comms end to end. Thank you.

Two things we are noting rather than asking for

- `image` **stays unmapped**, as you asked, until the templates are cleaned up. Understood and agreed - fixing the data beats us carrying a workaround.
- **C2's refresh trap.** Your warning about parallel refreshes spending a single-use token is exactly the shape of bug we would have shipped. One shared in-flight promise, and we will tell you when it is in.